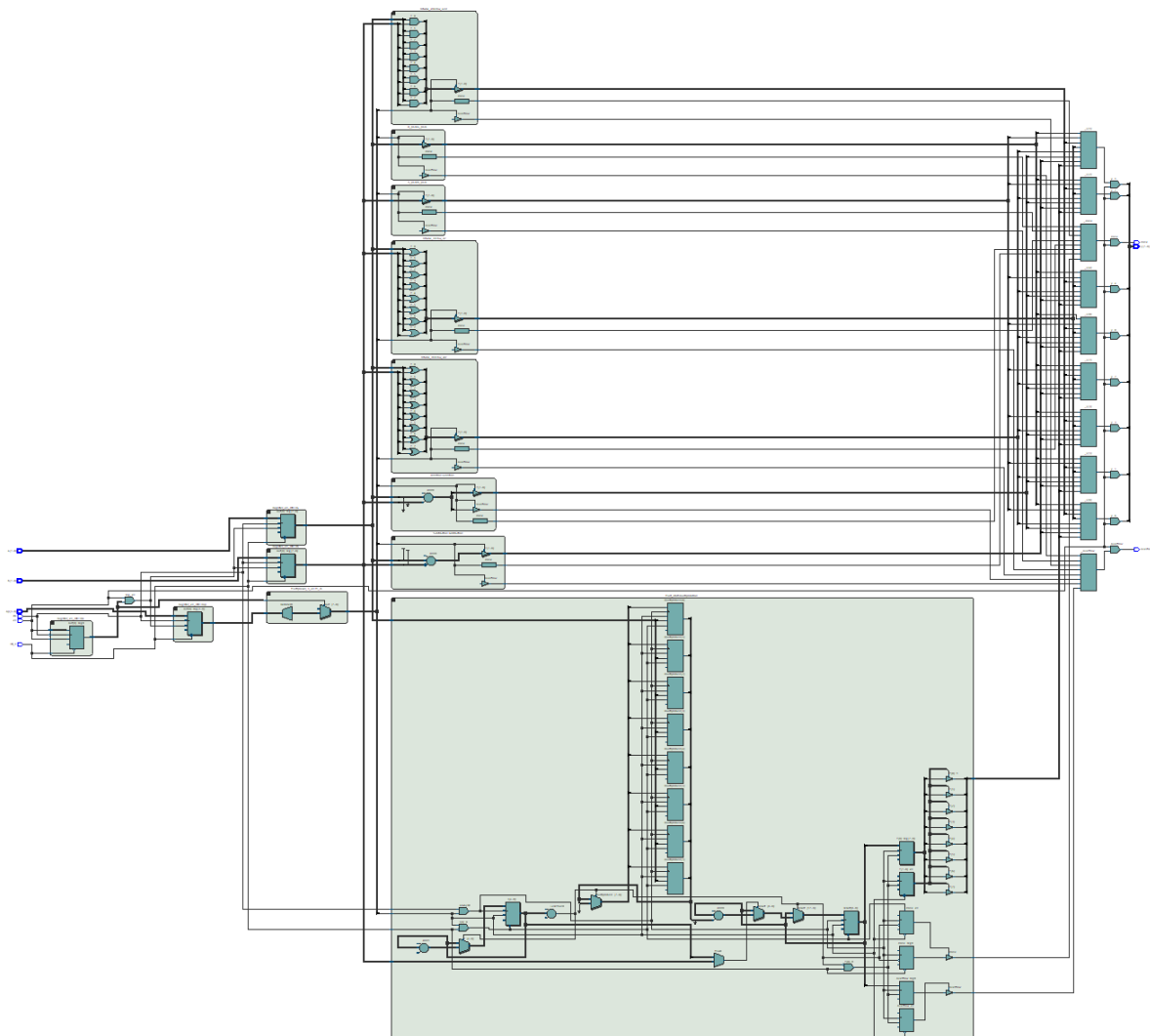


ECE 204 Design Project

8 Bit ALU

Nolan Kessler, Maxwell Fabian, John O'Connor

June 7, 2026



1 8 Bit ALU Functional Description

This 8 bit ALU accepts two 8 bit operands A and B, along with a 3 bit opcode (OP). The state of the ALU is controlled by an active high enable pin (EN), and can be reset using the active low reset pin (RST_N). As this ALU supports sequential operations which may take more than one clock cycle, the CLK input must be connected to a clock signal. The ALU provides one 8 bit result output (Y) an overflow flag (OVER) and a done flag (DONE). A table of the ALU's supported operations is given in table 1. A diagram of the inputs and outputs is shown in figure 1. A description of the function of each bus/pin is given in table 2.

Operation	Opcode	Clock Cycles	Overflow Condition	Description
A pass	0x0	1	N/A	Passes A to Y unmodified
B pass	0x1	1	N/A	Passes B to Y unmodified
Bitwise AND	0x2	1	N/A	Y is bitwise AND combination of operands A and B
Bitwise OR	0x3	1	N/A	Y is bitwise OR combination of operands A and B
Bitwise XOR	0x4	1	N/A	Y is bitwise XOR combination of operands A and B
Addition	0x5	1	$A + B > 255$	Y is the sum of operands A and B
Subtraction	0x6	1	$A + (\bar{B} + 1) > 255$	Y is the sum of A and the 2's complement inverse of B
Multiplication	0x7	9	$A * B > 255$	Y is the product of A and B

Table 1: Table of supported operations

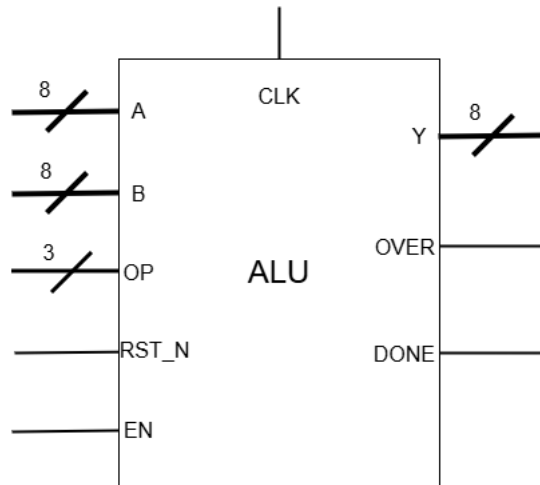


Figure 1: ALU Inputs and Outputs

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to mathematical or logical operation.
B	8	Second 8-Bit operand - input to mathematical or logical operation.
OP	3	Opcode - selects which operation to perform.
RST_N	1	Active low asynchronous reset - puts the ALU into initial ready state.
EN	1	Enable - starts ALU when high if in ready state. Must be high for ALU to output data.
Output		
Y	8	8 Bit result - output of the ALU operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or ALU is ready, high when operation is completed.

Table 2: ALU IO Functional Descriptions

1.1 ALU Operation

Before beginning an operation the ALU must be reset by setting RST_N low, after reset occurs the ALU is considered to be in the "ready" state. EN must be low for all rising edges of CLK while the ALU is in the ready state. Once the inputs on A, B, and OP are stable, EN can be set high to begin ALU operation. The rising edge of EN must occur when CLK is low. EN must remain high until after the next falling edge of CLK. At this point the ALU enters the "running" state. From this point on the value of EN can be changed without effecting the ALU (or the result when the ALU is done). The ALU will always output all zeros when EN is low. When DONE goes high the ALU is in the "done" state. The operation is finished, and results can be read from Y, and OVER. A timing diagram of a normal ALU operation is shown in figure 2.

2 Design and Implementation

This ALU is designed to utilize modular operations, to allow the design to allow for easy testing during the design process, and potential rapid addition of new functions in the future. These modular operations are then linked together with integrated together with structures to allow for the selection of operations, forming the complete ALU.

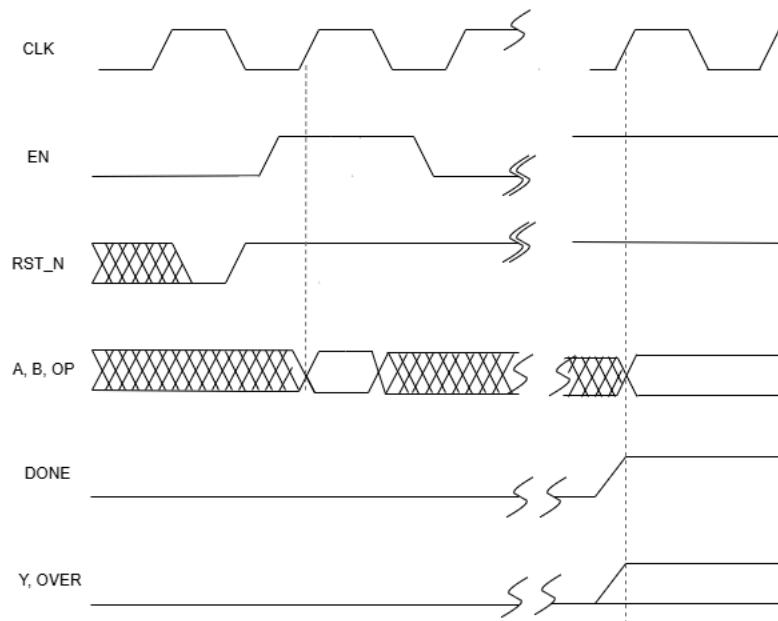


Figure 2: ALU Operation Timing Diagram

2.1 Operation Modules

Each operation module accepts the inputs, and supplies the outputs shown in table 3. All outputs of the modules are routed through tristate buffers - the outputs of an operation are floating unless its enable pin is high.

There are two primary types of operation modules utilized in this ALU, combinational logic operations, and sequential logic operations. For combinational logic operations, the CLK and RST_N inputs are not needed, and DONE is always high when the module is enabled. A diagram of such an operation module is shown in figure 3. Sequential logic operations utilize all inputs to perform operations that require more than one clock cycle. The only such operation in this ALU is multiplication. A diagram of this type of module is shown in 4.

The following sections will include a brief description of the implementation of each ALU operation, and a listing of its SystemVerilog code.

2.1.1 A Pass

Description here

Listing: a_pass.sv

```

1 module A_pass(
2     input logic [7:0] A,
3     input logic enable,
4     output logic [7:0] Y,
5     output logic done,
6     output logic overflow
7
8 );
9

```

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to operation.
B	8	Second 8-Bit operand - input to operation.
RST_N	1	Active low asynchronous reset - resets the module to inial ready state.
EN	1	Enable - enables the module. The module will only output data when this pin is high
Output		
Y	8	8 Bit result - output of the operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or is ready, high when operation is completed.

Table 3: Operation Module IO Descriptions

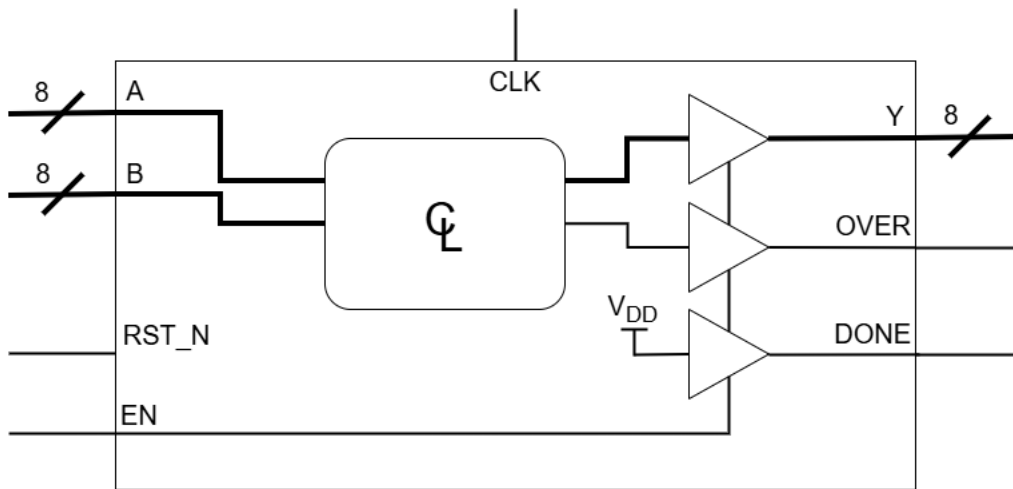


Figure 3: Schematic of a Combinational Operation Module

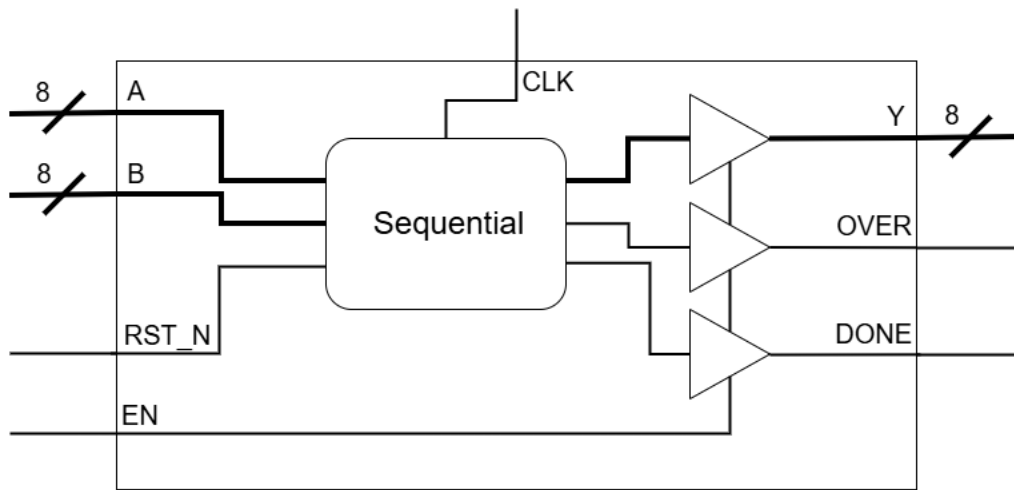


Figure 4: Schematic of a Sequential Operation Module

```

10     assign Y = enable ? A : 8'bz;
11     assign done = enable ? 1'b1 : 1'bz;
12     assign overflow = enable ? 1'b0 : 1'bz;
13
14
15
16 endmodule

```

2.1.2 B Pass

Description here

Listing: b_pass.sv

```

1 module B_pass(
2     input logic [7:0] B,
3     input logic enable,
4     output logic [7:0] Y,
5     output logic done,
6     output logic overflow
7 );
8
9
10     assign Y = enable ? B : 8'bz;
11     assign done = enable ? 1'b1 : 1'bz;
12     assign overflow = enable ? 1'b0 : 1'bz;
13
14
15 endmodule

```

2.1.3 Bitwise AND

Description here

Listing: bw_and.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the Bitwise AND in the ALU
7  */
8  module Bitwise_AND(
9      input logic [7:0] A, B, //8-bit Operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,    //8-bit Output
12     output logic done,       //Done Flag
13     output logic overflow    //Overflow Flag
14 );
15
16     //Assigns Y to the output of Bitwise AND when enable is high
17     assign Y = enable ? (A & B) : 8'bz;
18
19     //Simple Functions like this one have simple done and overflow flags
20     assign done = enable ? 1'b1 : 1'bz;
21     assign overflow = enable ? 1'b0 : 1'bz;
22
23 endmodule
```

2.1.4 Bitwise OR

Description here

Listing: bw_or.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the Bitwise OR in the ALU
7  */
8  module Bitwise_OR (
9      input logic [7:0] A, B, //8-bit Operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,    //8-bit Output
12     output logic done,       //Done Flag
13     output logic overflow    //Overflow Flag
14 );
15
16     //Assigns Y to the output of Bitwise OR when enable is high
17     assign Y = enable ? (A | B) : 8'bz;
18
19     //Simple Functions like this one have simple done and overflow flags
20     assign done = enable ? 1'b1:1'bz;
21     assign overflow = enable ? 1'b0:1'bz;
```

```
22
23 endmodule
```

2.1.5 Bitwise XOR

Description here

Listing: bw_xor.sv

```
1 module Bitwise_XOR (
2     input logic [7:0] A, B,
3     input logic enable,
4     output logic [7:0] Y,
5     output logic done,
6     output logic overflow
7 );
8
9
10     assign Y = enable ? (A ^ B) : 8'bz;
11     assign done = enable ? 1'b1 : 1'bz;
12     assign overflow = enable ? 1'b0 : 1'bz;
13
14 endmodule
```

2.1.6 Addition

Description here

Listing: addition.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the adder in the ALU
7  */
8  module Addition(
9      input logic [7:0] A, B, //8-bit operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,    //8-bit output
12     output logic overflow,   //Overflow Flag
13     output logic done        //Done Flag
14 );
15     //Assigns overflow as 9th bit to Y and takes adder when enable is high
16     assign {overflow, Y} = enable ? (A + B) : 9'bz;
17
18     //Done flag on enable high
19     assign done = enable ? 1'b1:1'bz;
```



```
20
21 endmodule
```

2.1.7 Subtraction

Description here

Listing: subtraction.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the Subtractor in the ALU
7  */
8  module Subtraction (
9      input logic [7:0] A, B, //8-bit Operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,     //8-bit Output
12     output logic done,        //Done Flag
13     output logic overflow     //Overflow Flag
14 );
15
16     //Assigns Y to the output of a subtractor when enable is high
17     assign Y = enable ? (A - B) : 8'bz;
18
19     //Simple Functions like this one have simple done and overflow flags
20     assign done      = enable ? 1'b1:1'bz;
21     assign overflow  = enable ? 1'b0:1'bz;
22
23 endmodule
```

2.1.8 Multiplication

Description here

Listing: mult.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources:
5      *https://en.wikipedia.org/wiki/Binary_multiplier#Single-cycle_multiplier
6
7      ↪ *https://stackoverflow.com/questions/63655634/8-bit-sequential-multiplier-using-add-and-shift
8  */
9  module Mult_8bit (
10     input logic [7:0] A, B, //8-bit Operand inputs
11     input logic clock,
```

```

11     input logic reset_n,
12     input logic enable, //Enable - sets output of operator to floating
13     output logic [7:0] Y, // 8-bit Output
14     output logic done, //Done Flag
15     output logic overflow //Overflow Flag
16 );
17
18 logic [8:0] result;
19 logic [7:0] multiplicand;
20 logic [4:0] i;
21
22
23     always @(posedge (clock && enable) or negedge reset_n or negedge(enable)) begin
24
25         if(~enable)begin
26             Y             <= 8'bz;
27             done          <= 1'bz;
28             overflow <= 1'bz;
29         end else begin
30
31             if(~reset_n) begin
32                 done          <= 1'b0;
33                 result        <= 0;
34                 multiplicand   <= A;
35                 i             <= 0;
36                 //trig_iterator <= 0;
37                 //trig_addstore <= 0;
38             end else begin
39
40                 if(i < 8) begin
41                     multiplicand <= multiplicand << 1;
42
43                     if(B[i] == 1) begin
44                         //trig_addstore <= 1'b1;
45                         result <= result + multiplicand;
46                     end
47                     i <= i + 1;
48                     //trig_iterator <= 1'b1;
49
50                 end else begin
51                     {overflow, Y} <= result;
52                     done          <= 1'b1;
53                 end
54
55                 //trig_addstore <= 0;
56                 //trig_iterator <= 0;
57             end
58         end
59     end
60
61 /*
62 CounterNbit #(N(7)) Iterator (
63     .clock(clock),
64     .reset_n(reset_n), .enable_n(trig_iterator),
65     .addBy(1),
66     .count(i)

```

```

67 );
68
69 CounterNbit #(N(8)) AddStore (
70     .clock(clock),
71     .reset_n(reset_n), .enable_n(trig_addstore),
72     .addBy(multiplicand),
73     .count(result)
74 );
75 );
76 */
77 endmodule
78
79

```

2.2 ALU Utility Modules

Three SystemVerilog modules were developed to be used within ALU operations or the ALU itself. These include a variable width asynchronously resettable register with enable, a 3 bit multiplexer with enable, and a variable width counter. The Implementation details and SystemVerilog listings for these modules are provided in the subsequent sections.

2.2.1 3 Bit Multiplexer

Description here

Listing: multiplexer_3_en.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: Nolan Kessler
4
5  Implements a three bit multiplexer with enable
6  */
7
8  module Multiplexer_3_en (
9      input logic [2 : 0] in,
10     input logic enable, //Output of multiplexer is 0 unless enable is high.
11     output logic [7 : 0] out
12 );
13     always_comb begin
14         if (enable) begin
15             case (in)
16                 3'b000: out = 16'b0000_0001;
17                 3'b001: out = 16'b0000_0010;
18                 3'b010: out = 16'b0000_0100;
19                 3'b011: out = 16'b0000_1000;
20                 3'b100: out = 16'b0001_0000;
21                 3'b101: out = 16'b0010_0000;
22                 3'b110: out = 16'b0100_0000;
23                 3'b111: out = 16'b1000_0000;
24             endcase

```

```

25     end
26     else begin
27         out = 4'b0;
28     end
29 end
30 endmodule

```

2.2.2 Variable Width Register

Description Here

Listing: register_en_rstn.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: Nolan Kessler
4  Sources: Based on my register implementation from Lab 5
5
6  Implements a parameterized width register with enable and active low reset.
7  */
8
9  module Register_en_rstn #(parameter WIDTH = 1) (
10     input logic clk,
11     input logic enable, //Enable - copy in to out on rising edge of clk only when high
12     input logic rst_n, //Active low reset - set out to zero on falling edge
13     input logic [WIDTH - 1 : 0] in,
14     output logic [WIDTH - 1 : 0] out
15 );
16
17     always_ff @(posedge clk, negedge rst_n) begin
18
19         if (rst_n == 0) begin
20             out <= 0;
21         end
22
23         else begin
24             if (enable != 0) begin
25                 out <= in;
26             end
27         end
28
29     end
30
31 endmodule

```

2.2.3 Counter

Description here

Listing: counter.sv

```
1 module CounterNbit #(parameter N = 3)(
2     input logic clock,
3     input logic reset_n, enable_n,
4     input logic [N:0] addBy,
5     output logic [N:0] count
6 );
7
8 logic [N:0] count_next;
9
10 always_comb begin //@(posedge clock || reset_n) begin
11     if (reset_n == 1'b0) begin
12         count_next <= count + addBy;
13     end else begin
14         count_next <= 0;
15     end
16 end
17
18 Register_en_rstn #(.WIDTH(N+1)) RegNBit (
19     .clk(clock),
20     .rst_n(reset_n), .enable(enable_n),
21     .in(count_next),
22     .out(count)
23 );
24
25 endmodule
26
```

3 Design Verification

4 Hardware Validation